

PIXIU TECH LLP

Robotics Lab Book

Class 11 | Integrated Systems, Data and Capstone Engineering

Guard a doorway with a beam of light, measure how wrong your own sensor is, and finish with a robot that solves a maze and remembers the way.

FIVE UNITS Introduction | Arduino IDE | Basic | Intermediate | Capstone

FIFTEEN SESSIONS One session = one class

THREE PROJECTS Laser Security - Ultrasonic Calibration - Maze Solver Robot

BOARD Arduino Uno (ATmega328P)

DELIVERABLES Working hardware, a written test report and a technical viva

THIS BOOK BELONGS TO _____

SCHOOL / SECTION _____

Contents

Fifteen sessions. One session is one class period.

Unit 1 - Introduction	3
Session 1 - Engineering a System, and What Is in Your Box	3
Session 2 - Measurement, Error and Uncertainty	5
Check Yourself - Unit 1	8
Unit 2 - The Arduino IDE	8
Session 3 - Inside the Board and Inside the Chip	8
Session 4 - From Typed Text to Running Machine Code	10
Session 5 - Data Acquisition and Non-Blocking Code	11
Check Yourself - Unit 2	14
Unit 3 - Basic Project: Laser Security System	14
Session 6 - Laser Diodes, LDRs and Optical Alignment	14
Session 7 - Latching Alarms and the LCD Counter	16
Session 8 - Reset, False Triggers and a Written Test Report	18
Check Yourself - Unit 3	21
Unit 4 - Intermediate Project: Ultrasonic Measurement and Calibration	21
Session 9 - Inside an Ultrasonic Transducer	21
Session 10 - Characterising Your Own Sensor	23
Session 11 - Filtering, Correction and Error Analysis	24
Check Yourself - Unit 4	27
Unit 5 - Capstone Project: Maze Solver Robot	27
Session 12 - Specification, Algorithms and the Left-Hand Rule	27
Session 13 - Subsystem Build and Independent Testing	29
Session 14 - State Machines and Path Memory	31
Session 15 - Optimisation, Final Runs and Technical Viva	33
Check Yourself - Unit 5	35
Extra Challenges and Twenty Revision Questions	35
Wiring Reference - All Three Projects	37
Words to Know	38
Answer Key	39
My Engineering Log	41

When you see	It means
Today you will	The three things this class is about. Read it before you start.
Behind the Scenes	How the component is built, and why it behaves as it does.
Try This	A short investigation to run at your bench.
Watch Out	A mistake almost everyone makes. Read it before you wire.
Dark box	Arduino code. Type it exactly - every bracket and semicolon matters.
Challenge	An extension worth attempting if your build is working early.

UNIT 1

Introduction

Sessions 1 and 2

This year the hardware is only half the work. You will be expected to specify before you build, measure your own error honestly, and defend your design decisions with evidence. That is the difference between making something work and engineering it.

By the end of this unit you will be able to:

- Write a requirement specification before starting a build.
- Name every component in your kit and say what it does.
- Distinguish accuracy from precision, and error from uncertainty.
- Explain why every measurement is a range rather than a single number.

SESSION 1

one class

Engineering a System, and What Is in Your Box

Today you will

- Learn what a requirement specification is and why it comes first.
- Learn the names of all sixteen components in your kit.
- Agree the rules of the robotics lab.

Anybody can wire a sensor to a buzzer. What separates an engineer is the order of the work. An engineer writes down what the machine must do **before** deciding how to do it, because otherwise there is no way to tell whether the finished thing is any good.

That written statement is a **requirement specification**. A good requirement is testable: somebody who did not build the machine must be able to read it, run a test, and say yes or no. Compare these two attempts at the same idea.

A poor requirement	Why it fails	A testable requirement
The alarm should be fast	How fast? Measured how?	The alarm shall sound within 500 ms of the beam being broken.
It should not give false alarms	Not measurable at all	The system shall produce no alarms during a 5 minute undisturbed test.
It should be accurate	Accurate to what?	Readings shall be within 1 cm of a tape measure between 10 and 150 cm.

Try This

Write three testable requirements for a school bell system: one about timing, one about volume, one about what happens during a power cut. Swap with another group and try to find a way each of their requirements could technically be met while still producing a useless bell. That exercise is how real specifications get tightened.

1.1 Open the Box

Lay everything out before connecting anything. The label colour on each tile tells you roughly which unit first needs that part. Two items deserve immediate respect: the laser module, which must never be pointed at anybody, and the battery pack, which must be disconnected whenever you rewire.



Figure 1.1 - Your kit. Learn these sixteen names before the next class.

Part	What it does	First used in
Arduino Uno board	The controller. It stores your program and runs it indefinitely.	Session 3
USB A-B cable	Carries the program in and powers the board during development.	Session 3
400-point breadboard	Solderless prototyping base.	Session 2
Jumper wires	Interconnections: male-male, male-female, female-female.	Session 2
5mm LEDs	Status indication: armed, alarm, ready.	Session 2
220 ohm and 10K resistors	Current limiting, and pull-downs for switch inputs.	Session 2
5V laser dot diode module	Emits the beam your alarm watches.	Session 6
LDR light sensor module	The receiver that catches the beam.	Session 6
16x2 LCD display	Shows the intrusion count and, later, live distance.	Session 7
10K potentiometer	Sets display contrast.	Session 7
Buzzer module	The alarm output.	Session 7
2-pin push switch	Clears the latched alarm.	Session 8
HC-SR04 ultrasonic sensor	The instrument you will characterise in Unit 4.	Session 9
IR sensor modules (three)	Read the maze: left, centre and right.	Session 12
L298N motor driver	Power stage between the controller and the motors.	Session 13

Part	What it does	First used in
Chassis with motors, wheels, caster and battery holder	The robot platform and its power source.	Session 13

1.2 Rules of the Robotics Lab

- 1 Always unplug the USB cable before changing any wire.
- 2 **Never aim the laser at anybody's eyes.** Mount it fixed, at bench level, pointing away from where people sit or walk.
- 3 Check polarity on every LED and module before applying power.
- 4 Never join 5V directly to GND. That is a short circuit.
- 5 Never connect a motor directly to an Arduino pin - the pin will be destroyed.
- 6 **Disconnect the battery pack before rewiring,** and at the end of every session.
- 7 Lift the robot off the floor before uploading, so it cannot drive away mid-upload.
- 8 Record every test in your engineering log, including the ones that failed.

Watch Out

Rule 2 is not negotiable. Even a low-power laser module is a collimated beam - all its energy arrives on a spot smaller than a pinhead, and your eye focuses it smaller still. The safe habit is simple: fix the module down before you power it, and keep the beam below eye level at all times.

SESSION 2

one class

Measurement, Error and Uncertainty

Today you will

- Distinguish accuracy from precision.
- Understand systematic error and random error.
- Express a measurement properly, as a value with an uncertainty.

Unit 4 of this book is an entire project about how wrong a sensor is. Before that can mean anything, you need the vocabulary - and most people, including many adults, use these words interchangeably when they mean completely different things.

Term	What it means	A sensor with the problem
Accuracy	How close a reading is to the true value	Reads 25 cm when the true distance is 30 cm
Precision	How closely repeated readings agree with each other	Reads 30, 34, 27, 33 cm for the same fixed object
Systematic error	A consistent offset or drift, the same every time	Always reads 3 cm too far - correctable
Random error	Unpredictable scatter around the true value	Wobbles a centimetre either way - reducible by averaging
Resolution	The smallest change the system can detect at all	Cannot tell 30.0 cm from 30.2 cm

The crucial insight is that accuracy and precision are independent. A sensor can be precise and wrong - giving you 25.0 cm every single time when the truth is 30. That is far more dangerous than a scattered sensor, because the consistency makes it look trustworthy. It is also, fortunately, the easier fault to fix: a systematic error can be measured once and corrected in software forever.

Behind the Scenes

This is why professional instruments carry a calibration certificate rather than a claim of perfection. The certificate does not say the instrument is right. It says how wrong it is allowed to be, and when it was last checked against a standard. Your Unit 4 project will produce exactly that kind of document for your own HC-SR04.

2.1 Writing a Measurement Properly

A single number is never a complete measurement. *The distance is 42 cm* hides the question a reader actually needs answered: how sure are you? The professional form is a value with an **uncertainty**: **42 plus or minus 2 cm**. That says the true value is very likely somewhere between 40 and 44.

You will estimate uncertainty the honest way: take several readings of the same fixed thing, and look at the spread. If ten readings of a wall at exactly 50 cm range from 48 to 52, your uncertainty is about 2 cm. No theory required - just data.

Try This

Measure the width of your bench ten times with a ruler, each person in the group taking a turn without looking at the previous answers. Write down all ten. Find the mean, the largest and the smallest. Now express the width as a value plus or minus an uncertainty. You have just performed the same procedure a national standards laboratory uses, only with cheaper equipment.

2.2 The Electrical Foundation

Everything in this book rests on one relationship, **Ohm's law**: $V = I \times R$. Voltage in volts equals current in amps multiplied by resistance in ohms. It is worth using it once properly rather than treating component values as magic.

Your LED drops about 2 volts across itself, so with a 5 volt supply the resistor must absorb the remaining 3 volts. With a 220 ohm resistor the current is 3 divided by 220, about 13.6 milliamps - comfortably within both the LED's rating and the 20 milliamp limit of an Arduino pin. Choose 22 ohms and you would demand 136 milliamps, and something would be destroyed.

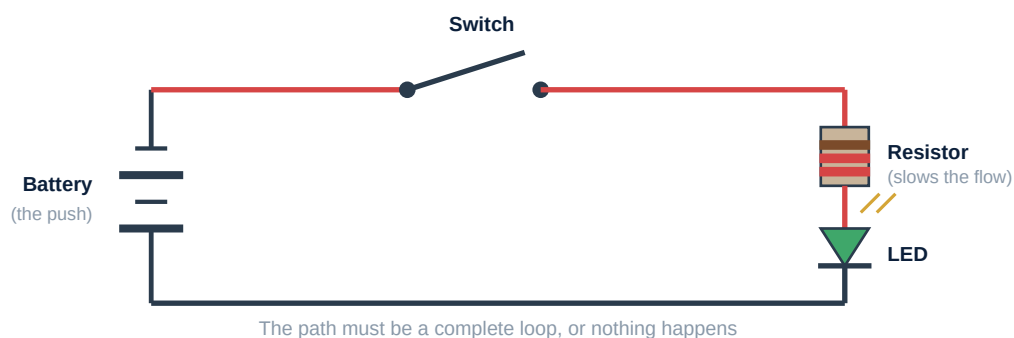


Figure 1.2 - A simple circuit: battery, switch, resistor and LED

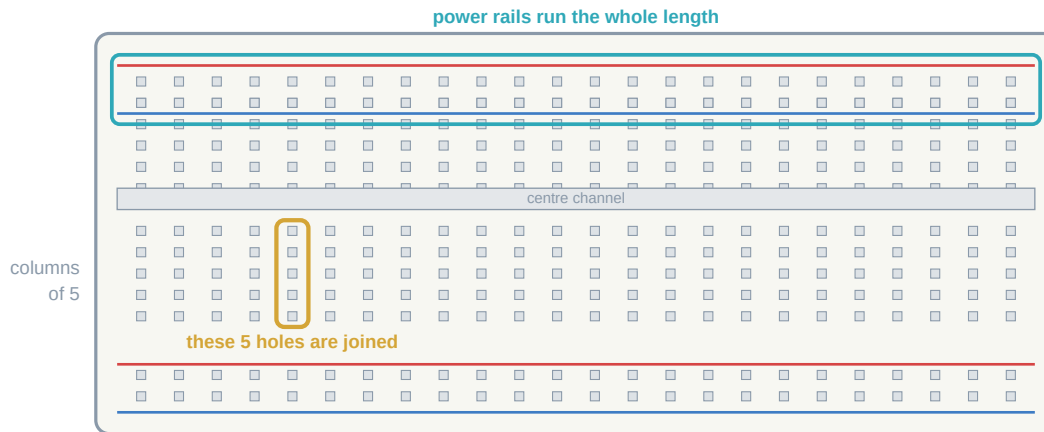


Figure 1.3 - Inside a breadboard: which holes are secretly connected

- The **long side rails** marked + and - run the full length. Use them for power and ground.
- Central holes are joined in **columns of five**, vertically only.
- The **centre channel** isolates the two halves, which is why chips straddle it.
- Two legs in the same column are connected; in different columns they are not.

New word	What it means
Requirement specification	A written, testable statement of what a system must do.
Accuracy	How close a reading is to the true value.
Precision	How closely repeated readings agree with each other.
Systematic error	A consistent offset or drift that can be corrected.
Random error	Unpredictable scatter, reducible by averaging.
Uncertainty	The range within which the true value very likely lies.

CHECK WHAT YOU LEARNT

Check Yourself - Unit 1**A. Choose the correct answer**

1. A requirement is only useful if it is
(a) ambitious (b) testable (c) short (d) technical
2. A sensor that always reads 3 cm too far has
(a) random error (b) systematic error (c) poor precision (d) no error
3. A sensor giving 30, 34, 27, 33 cm for one fixed object has poor
(a) accuracy (b) precision (c) resolution (d) range
4. Random error is best reduced by
(a) a correction factor (b) averaging several readings (c) a bigger resistor (d) recalibrating
5. 42 plus or minus 2 cm means the true value very likely lies between
(a) 40 and 42 (b) 40 and 44 (c) 42 and 44 (d) 38 and 46
6. With 3 volts across a 220 ohm resistor, the current is about
(a) 1.4 mA (b) 13.6 mA (c) 136 mA (d) 660 mA

B. Fill in the blanks

1. Ohm's law is written as $V = \text{_____} \times \text{_____}$.
2. A consistent offset that is the same every time is called _____ error.
3. The smallest change a system can detect is its _____.
4. A calibration certificate states how _____ an instrument is allowed to be.

C. Answer in one or two lines

1. Rewrite 'the robot should be fast' as a testable requirement.
2. Explain why a precise but inaccurate sensor is more dangerous than a scattered one.
3. Describe how you would estimate the uncertainty of a measurement using only data.
4. Work out the current through a 330 ohm resistor with 3 volts across it.

UNIT 2

The Arduino IDE

Sessions 3, 4 and 5

You cannot debug what you cannot picture. This unit opens the board, opens the chip, and follows your code from the letters you type to the machine instructions that actually execute.

By the end of this unit you will be able to:

- Name the parts of the Arduino Uno board and the blocks inside the ATmega328P.
- Describe every stage between pressing Upload and the program running.
- Explain what the ADC measures and calculate its resolution.
- Write non-blocking code using `millis()` instead of `delay()`.

SESSION 3

one class

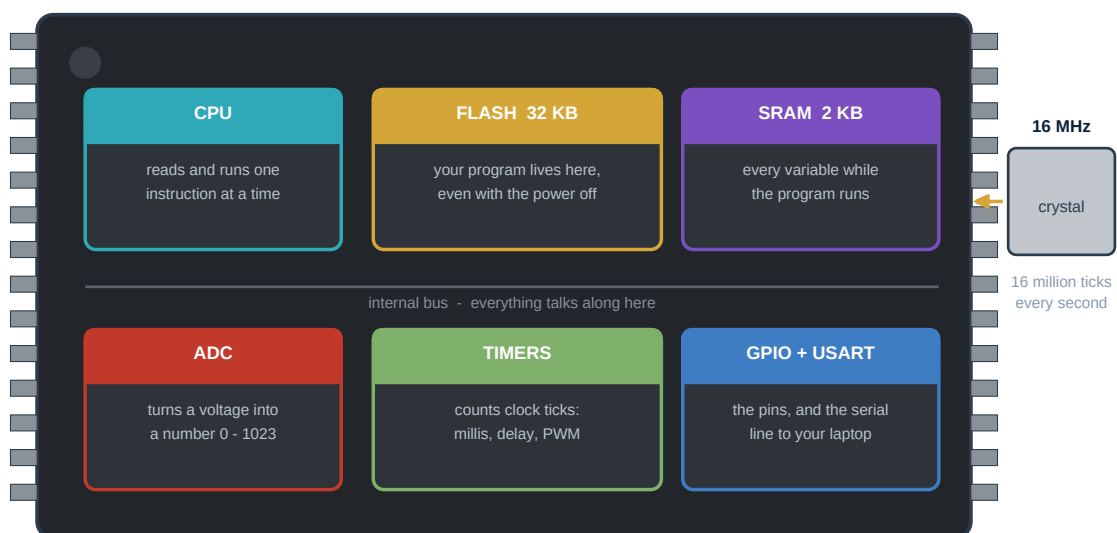
Inside the Board and Inside the Chip

Today you will

- Identify every major component on the Arduino Uno board.
- Learn what the ATmega328P contains and how much of each.
- Confirm the whole toolchain works with a test upload.

The blue board is not the computer. It is a **support system** for one chip, and every other part on that PCB exists to make the chip convenient for a human to use.

Component on the board	Its job
ATmega328P - the long black chip	The microcontroller. This is the actual computer.
16 MHz quartz crystal	Sets the clock: sixteen million ticks every second.
Second, smaller chip near USB	Converts USB signalling into the plain serial the ATmega understands.
Three-legged regulator near the jack	Turns 7 to 12V from a battery into a steady 5V.
Electrolytic capacitors	Smooth out dips in the supply when something draws a sudden gulp.
Headers	The pins D0 to D13 and A0 to A5 you plug into.
Reset button	Restarts the program from the first instruction.

ATmega328P - the chip inside the Arduino Uno

Everything you write ends up as instructions the CPU fetches from flash, one at a time, sixteen million times a second.

Figure 2.1 - Behind the blue board: what is actually doing the work

Block inside the chip	Size	What it does	Lost on power off?
Flash	32 KB	Holds your compiled program	No
SRAM	2 KB	Holds every variable while running	Yes
EEPROM	1 KB	Small data you want kept	No
CPU	8-bit	Executes one instruction at a time	-
ADC	10-bit	Converts a voltage to 0 - 1023	-
Timers	three	Drive millis(), delay() and PWM	-

Behind the Scenes

Two kilobytes of SRAM is a genuine constraint, not a curiosity. Every variable, every array and every text message lives there simultaneously. Your maze solver will store its path in an array - and if you make that array too large, the program will not warn you, it will simply start behaving inexplicably as the memory overflows into itself. Budget your SRAM the way you would budget money.

SESSION 4

one class

From Typed Text to Running Machine Code

Today you will

- Follow the four stages of compiling and uploading.
- Understand what the bootloader is and why the board resets.
- Diagnose an error message by the stage it came from.

Between pressing Upload and your program running, four distinct things happen in about two seconds. Knowing which stage a failure came from turns a baffling red message into a two-second diagnosis.

- 1 Compiling.** Your C++ text is translated into ATmega machine instructions. Any syntax error stops everything here and nothing is transmitted.
- 2 Linking.** The libraries you included are combined with your code into a single file of hexadecimal machine code - the .hex file.
- 3 Resetting.** The IDE pulses the board's reset line. The chip restarts and runs a small resident program called the **bootloader** for about one second.
- 4 Writing.** The bootloader listens on the serial line, receives the .hex file in packets, and writes them into flash. It then jumps to your program, which begins immediately.

Behind the Scenes

The bootloader occupies about half a kilobyte of your flash and costs you a second of start-up on every reset. Professional products remove it entirely and program the chip directly through its ISP header, which is why a commercial device boots instantly. What the bootloader buys you is the ability to program the board with nothing more than a USB cable - a very good trade for a laboratory.

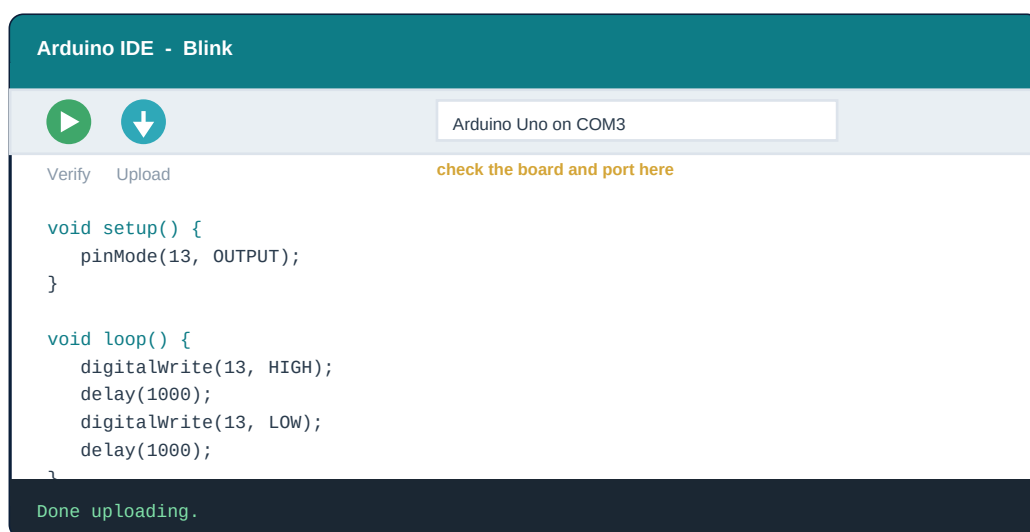


Figure 2.2 - Verify checks your code; Upload sends it to the board

Message	Stage that failed	Diagnosis
expected ';' before ...	Compiling	Missing semicolon on the line above
was not declared in this scope	Compiling	Typo, wrong capitals, or a missing #include
no matching function for call	Compiling	Wrong number or type of arguments
Sketch too big	Linking	Program exceeds 32 KB of flash
Low memory available	Linking	Variables and strings exceed comfortable SRAM
Board at COM__ is not available	Resetting	Wrong port, or a charge-only USB cable
programmer is not responding	Writing	Wrong board selected, or the bootloader was interrupted

SESSION 5

one class

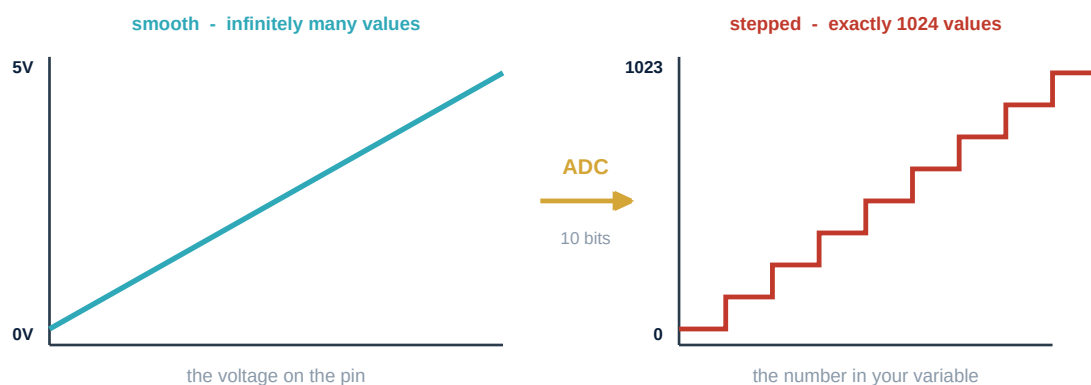
Data Acquisition and Non-Blocking Code

Today you will

- Understand what the ADC measures and calculate its resolution.
- Learn why delay() is unacceptable in a real system.
- Write a non-blocking loop using millis().

An analog pin measures exactly one quantity: **voltage**. Everything else - light, distance, temperature - reaches the Arduino only after some circuit has converted it into a voltage first. Inside the chip, the Analog to Digital Converter compares that voltage against a reference and reports a whole number.

Analog to Digital Conversion



$$\text{one step} = 5V / 1024 = \text{about } 4.9 \text{ millivolts}$$

a change smaller than that is invisible to the Arduino. That is its resolution.

Figure 2.2 - Every analogRead() is a measurement rounded to the nearest step

The converter is **10-bit**: 2 to the power 10, giving 1024 distinct values from 0 to 1023. Across a 5 volt reference each step is 5 divided by 1024, about 4.9 millivolts. That is the resolution, and no amount of clever code can see a change smaller than one step.

Behind the Scenes

The ADC uses **successive approximation**. It sets an internal reference to half of full scale and asks whether the input is above it, then halves the remaining range and asks again, ten times over - a binary search executed in hardware. Each conversion takes roughly 100 microseconds. In a loop polling several sensors that is not free, and in Unit 4 you will care about it.

5.1 Why delay() Is Not Acceptable

`delay(1000)` does not schedule anything. It halts the processor completely for a second. During that second your machine cannot read a sensor, cannot notice a button, cannot update a display and cannot stop a motor. In a toy that is a nuisance. In a security system or a moving robot it is a defect.

The alternative is to stop thinking in terms of *waiting* and start thinking in terms of *elapsed time*. `millis()` reports how many milliseconds the board has been running. Record the time an event happened, and on every pass of the loop ask whether enough time has gone by. The loop never stops turning.

The non-blocking pattern you will use all year

```
unsigned long lastBeep = 0;
unsigned long lastRead = 0;

void loop() {
    unsigned long now = millis();

    if (now - lastRead > 100) {        // sample the sensor ten times a second
        lastRead = now;
        // ... take a reading here ...
    }

    if (now - lastBeep > 500) {        // and blink the alarm twice a second
        lastBeep = now;
        // ... toggle the buzzer here ...
    }

    // the loop keeps running - nothing is ever frozen
}
```

Watch Out

Always declare a `millis()` variable as `unsigned long`. An ordinary `int` overflows after about 32 seconds and an ordinary `long` after 24 days; `unsigned long` lasts roughly 49 days. Writing `now - lastRead` rather than `now > lastRead + 100` also keeps working correctly even at the moment the counter wraps around.

5.2 The Reverse Operation: PWM

If the ADC turns a voltage into a number, PWM does close to the opposite. A digital pin can only be fully on or fully off, so the chip switches it about 490 times a second and varies how much of each cycle is spent on. The **average** voltage changes even though the pin never sits anywhere in between.

PWM - switching so fast that your eye sees an average`analogWrite(pin, 64)`

dim

`analogWrite(pin, 128)`

half bright

`analogWrite(pin, 255)`

almost full

The pin is still only ON or OFF. It is the amount of time spent ON that changes.

Only pins marked with ~ can do this: D3, D5, D6, D9, D10 and D11.

Only the six pins driven by the timers can do this - D3, D5, D6, D9, D10 and D11, all marked with a small ~. In your capstone, `analogWrite(ena, 120)` means *hold the enable pin on for about 47 percent of each cycle*, and the motor's own inertia smooths the rest.

New word	What it means
ATmega328P	The microcontroller that is the real computer on the board.
Bootloader	A resident program that receives your code and writes it to flash.
ADC	The circuit converting a voltage to a number.
Successive approximation	The binary-search method the ADC uses.
Resolution	The smallest change a measuring system can detect.
Non-blocking	Code that waits without halting everything else.

CHECK WHAT YOU LEARNT

Check Yourself - Unit 2**A. Choose the correct answer**

- The actual computer on an Arduino Uno is
(a) the USB chip (b) the ATmega328P (c) the regulator (d) the crystal
- Variables live in
(a) flash (b) SRAM (c) EEPROM (d) the ADC
- A 10-bit ADC reports
(a) 10 values (b) 100 values (c) 1024 values (d) 1023 volts
- One ADC step across a 5V reference is about
(a) 4.9 microvolts (b) 4.9 millivolts (c) 49 millivolts (d) 0.5 volts
- The ADC finds its answer by
(a) counting up from zero (b) successive approximation (c) measuring current (d) guessing
- A `millis()` variable must be declared as
(a) `int` (b) `float` (c) `unsigned long` (d) `boolean`

B. Fill in the blanks

- The _____ receives your code and writes it into flash.
- The ATmega328P has _____ KB of flash and _____ KB of SRAM.
- `delay()` is unacceptable in a real system because it _____ the processor.
- Each ADC conversion takes roughly _____ microseconds.

C. Answer in one or two lines

- Describe the four stages between pressing Upload and your program running.
- A pin sits at 1.8 volts. What will `analogRead()` return? Show your working.
- Explain why `now - lastRead` is safer than `now > lastRead + 100`.
- Give one concrete example of a fault caused by using `delay()` in a moving robot.

UNIT 3

Basic Project: Laser Security System

Sessions 6, 7 and 8

A beam of light across a doorway, a receiver that notices when it breaks, and a counter that remembers every intrusion. Simple to describe, and full of the engineering decisions that separate a demonstration from a product.

By the end of this unit you will be able to:

- Explain how a laser diode produces a collimated beam.
- Determine a detection threshold from measured data rather than trial and error.
- Implement latching so that a fast event is never missed.
- Produce a written test report with false-positive and false-negative counts.

SESSION 6

one class

Laser Diodes, LDRs and Optical Alignment

Today you will

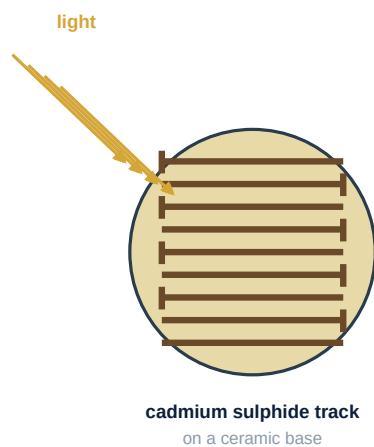
- Learn how a laser differs from an ordinary LED.
- Understand what is inside an LDR and why it responds to light.
- Align the beam and record threshold data.

A laser diode starts life as an ordinary light-emitting junction, but two additions transform it. First, the semiconductor is built as an **optical cavity** - mirrored at both ends, so light bounces back and forth and stimulates more light of exactly the same wavelength and exactly in step. Second, a tiny lens at the front **collimates** the output into a narrow parallel beam instead of letting it spread.

The result is what makes the whole project possible: an ordinary LED at three metres would spread into a dim patch, while a laser still arrives as a bright dot small enough to fall entirely on a sensor the size of your fingernail.

	Ordinary LED	Laser diode
Wavelengths emitted	a spread of colours	essentially one
Phase of the waves	random	in step with each other
Beam spread	wide, tens of degrees	narrow, fractions of a degree
Useful range for this project	a few centimetres	several metres
Eye safety	harmless	never look into the beam

The receiver is your LDR. Under its clear window is a zig-zag track of **cadmium sulphide** printed on ceramic. The zig-zag makes the conducting path long and narrow, which magnifies the effect you are about to exploit.



What the light actually does

In the dark

electrons stay locked in place
resistance high, about 1,000,000 ohms

In bright light

photons knock electrons free
resistance low, a few hundred ohms

This is called photoconductivity. Notice what the sensor does not do: it never measures brightness in any unit. It only changes its resistance, and the module turns that into a voltage your Arduino can read.

Figure - Inside an LDR: light knocks electrons loose and current flows more easily

Behind the Scenes

In darkness the electrons in cadmium sulphide are bound in place and the resistance can reach a million ohms. Each arriving photon carries just enough energy to free one electron, and every freed electron makes the material a little more conductive. Under a laser spot the resistance collapses to a few hundred ohms. The effect is called **photoconductivity**. Note the consequence: an LDR is slow, taking tens of milliseconds to settle - fine for a doorway, useless for high-speed counting.

Try This

Fix the laser and the LDR facing each other about 40 cm apart, both clamped or taped down. Print `analogRead(A0)` continuously. Record the value with the beam landing squarely on the sensor, then with your hand interrupting it, then with the room lights off, and finally with a torch shone at the LDR. Those four numbers decide your threshold - and the last one will tell you how easily your system can be fooled.

Condition	Reading on A0	What it tells you
Beam landing on the sensor	_____	the armed value
Beam interrupted by a hand	_____	the alarm value
Room lights off, beam on	_____	how much ambient light contributes
Torch shone at the LDR, beam broken	_____	how a defeat attempt would look
Threshold you chose	_____	justify it from the four above

SESSION 7

one class

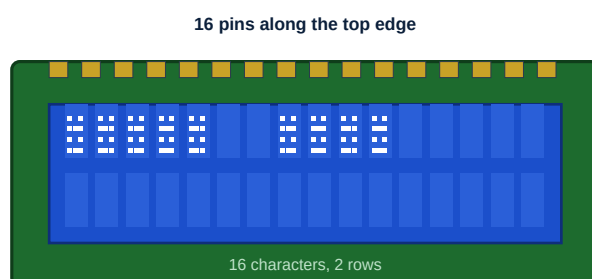
Latching Alarms and the LCD Counter

Today you will

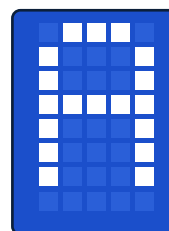
- Wire the laser, LDR, display, buzzer and reset switch.
- Understand why a fast event needs latching.
- Show a running intrusion count on the display.

Before wiring, it is worth knowing what the display in front of you actually is. The name sounds like a contradiction: how can something be both liquid and crystal? A liquid crystal is a genuine fourth state of matter - its long molecules flow freely like a liquid, yet they also line up with one another like a crystal, and that alignment can be changed by a small voltage.

Inside a 16x2 character LCD



a 5 x 8 grid of dots per character



the HD44780 controller stores the dot pattern for 208 characters

Why it is called liquid crystal

Between two sheets of glass sits a liquid whose long molecules normally line up in a twist. That twist rotates light passing through, so it escapes the far filter and the pixel looks pale. Apply a small voltage and the molecules straighten. The light no longer twists, the far filter blocks it, and that dot goes dark. No dot makes its own light - the backlight behind does that, which is why an LCD needs its LED backlight and why the contrast pot changes what you can read.

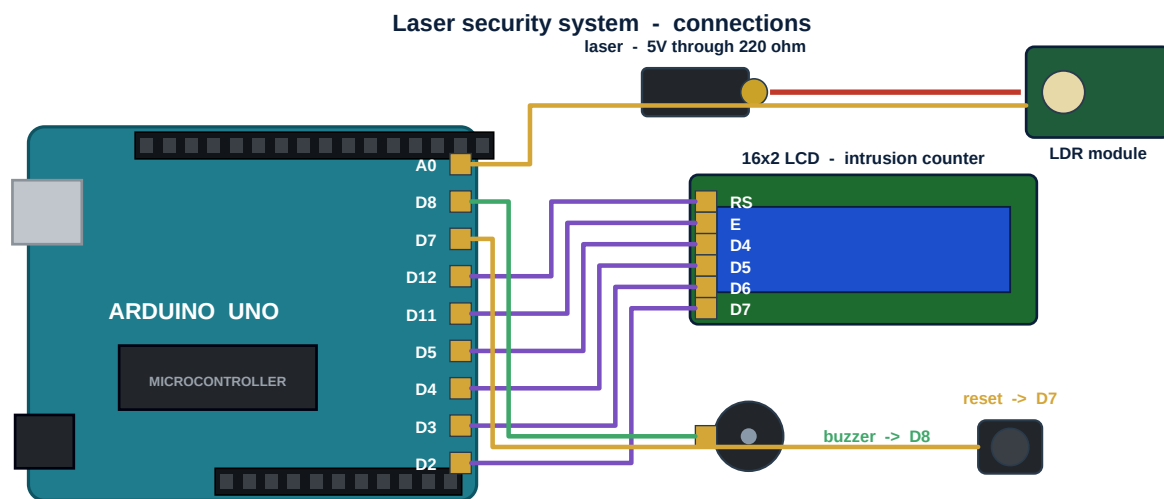
Behind the Scenes

An LCD dot produces **no light of its own** - it only blocks or passes light from the backlight behind it. That is why the display is unreadable if the backlight fails, why the contrast potentiometer matters so much, and why an LCD consumes so little power compared with a screen that generates its own light. The black blob on the back is an **HD44780** controller, the industry standard since the 1980s: it stores the dot patterns for 208 characters and refreshes every dot continuously, so you only ever send it a letter.

LCD pin	Name	Purpose
1, 2	VSS, VDD	Ground and 5V
3	V0	Contrast - this is what the potentiometer sets
4	RS	Register Select: is this byte a character or a command?
5	RW	Read or Write. Tied to ground - we only ever write.
6	E	Enable. A pulse means 'take the data now'.
7 to 14	D0 to D7	Data. In 4-bit mode only D4 to D7 are used.
15, 16	A, K	LED backlight. A to 5V through 220 ohm.

Behind the Scenes

Why use four data lines instead of eight? Because eight would consume eight Arduino pins and leave nothing for your sensors. In **4-bit mode** each byte is sent as two halves. It is half the speed - and entirely irrelevant, since the display needs updating a few times a second while the chip runs sixteen million times faster.



The laser is powered continuously. The Arduino only watches the LDR.

Figure 3.2 - A beam across a doorway, and a counter that remembers

Component	Arduino pin	Note
Laser module +	5V through 220 ohm	Powered continuously, not switched
Laser module -	GND	
LDR module AO	A0	The beam intensity
Buzzer +	D8	The alarm
Push switch	D7, with 10K to GND	Clears the latch
LCD RS / E	D12 / D11	Control lines
LCD D4-D7	D5, D4, D3, D2	Data, in that order
LCD V0	Potentiometer wiper	Contrast
LCD VSS, RW, K	GND	
LCD VDD, A	5V (A through 220 ohm)	Power and backlight

Now the design decision that matters. Somebody walking briskly through the beam interrupts it for perhaps eighty milliseconds. If your loop happens to be busy writing to the display at that instant, a simple if statement would miss the event entirely - and an intruder detector that misses intruders is worthless.

The answer is **latching**. A boolean records that the beam was broken. Once set, only the reset button can clear it. The machine has stopped merely reacting and started remembering.

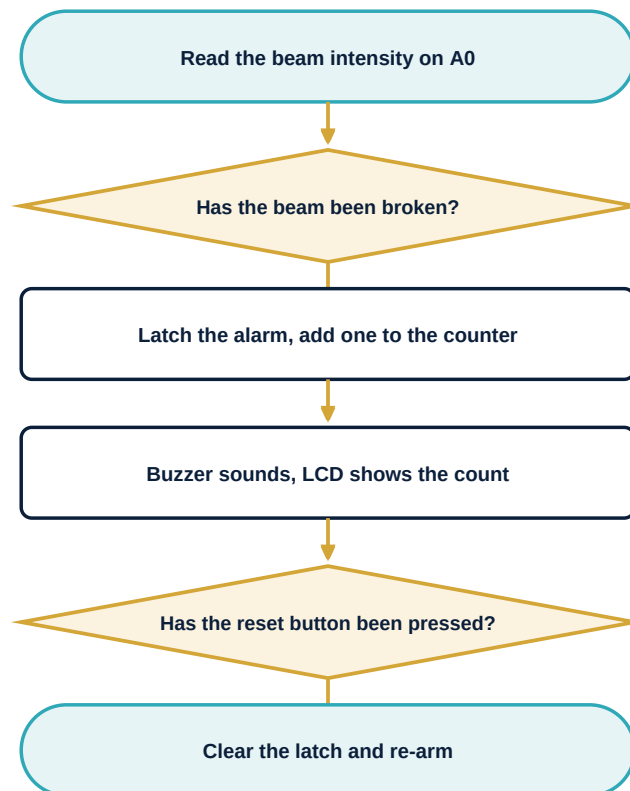


Figure 3.3 - Detect, latch, count, and wait for a human

SESSION 8

one class

Reset, False Triggers and a Written Test Report

Today you will

- Complete the program with debounced reset and timestamps.
- Run a formal test and count false positives and negatives.
- Write a one-page test report.

LaserSecuritySystem.ino

```

#include <LiquidCrystal.h>

LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

int ldrPin = A0, buzzer = 8, resetBtn = 7;
int threshold = 500;           // set this from YOUR measured data

boolean alarmLatched = false;
int intrusions = 0;
unsigned long lastEvent = 0;
unsigned long lastBeep = 0;
boolean beepState = false;

void setup() {
  pinMode(buzzer, OUTPUT);

```

```

pinMode(resetBtn, INPUT);
lcd.begin(16, 2);
lcd.print("System armed");
Serial.begin(9600);
}

void showStatus() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print(alarmLatched ? "INTRUSION!" : "System armed");
  lcd.setCursor(0, 1);
  lcd.print("Count: ");
  lcd.print(intrusions);
  if (intrusions > 0) {
    lcd.print(" ");
    lcd.print((millis() - lastEvent) / 1000); // seconds since last
    lcd.print("s");
  }
}

void loop() {
  unsigned long now = millis();
  int beam = analogRead(ldrPin);

  if (beam < threshold && !alarmLatched) { // beam broken, first time
    alarmLatched = true;
    intrusions = intrusions + 1;
    lastEvent = now;
    Serial.print("Intrusion ");
    Serial.print(intrusions);
    Serial.print(" at ");
    Serial.println(now);
    showStatus();
  }

  if (digitalRead(resetBtn) == HIGH) { // only a human clears it
    alarmLatched = false;
    digitalWrite(buzzer, LOW);
    showStatus();
    delay(300); // simple debounce
  }

  if (alarmLatched && now - lastBeep > 200) {
    lastBeep = now;
    beepState = !beepState;
    digitalWrite(buzzer, beepState); // non-blocking beeping
  }

  if (!alarmLatched && now - lastEvent > 1000 && now % 1000 < 20) {
    showStatus(); // refresh the idle display
  }
}

```

Three professional details are hiding in that code. The buzzer beeps using `millis()` rather than `delay`, so the system can still detect a second intrusion while it is already alarming. The condition includes `&& !alarmLatched` so one continuous interruption is counted once, not five hundred times. And every event is timestamped to the serial line, which is what turns a demonstration into evidence.

8.1 Testing Formally

A detector can fail in two distinct ways and they are counted separately, because their consequences differ completely. A **false positive** is an alarm with no intruder - annoying, and eventually fatal to trust. A **false negative** is an intruder with no alarm - the system has failed at the only job it had.

Test	Runs	Alarms raised	Result
Undisturbed for 5 minutes	1	_____	0 expected
Hand passed through the beam	10	_____	10 expected
Object passed quickly, under 100 ms	10	_____	10 expected
Room lights switched off and on	5	_____	0 expected
Torch shone at the LDR, beam broken	5	_____	note what happens
False positives		_____	
False negatives		_____	

Behind the Scenes

The torch test is the interesting one. If a bright torch on the LDR keeps the reading above your threshold even while the beam is blocked, then your alarm can be defeated by anybody with a phone. Real beam-break sensors defeat this by **modulating** the beam - pulsing it at a known frequency and accepting only light that pulses in the same rhythm. Steady light, however bright, is ignored.

Challenge

Write the one-page test report: a paragraph of purpose, your threshold and the data that justified it, the table above completed, the false positive and negative counts, and two sentences on the limitations you found. This document is assessed alongside the hardware.

New word	What it means
Collimated	Formed into a narrow parallel beam.
Optical cavity	Mirrored ends that make a laser's light coherent.
Photoconductivity	Light freeing electrons and lowering resistance.
Latching	Staying on once triggered, until deliberately cleared.
Debounce	Ignoring the brief contact chatter of a switch.
Modulation	Pulsing a signal at a known rate so it cannot be imitated.

CHECK WHAT YOU LEARNT

Check Yourself - Unit 3**A. Choose the correct answer**

1. A laser differs from an LED mainly because its light is
(a) brighter (b) coherent and collimated (c) warmer (d) invisible
2. In darkness the resistance of an LDR is
(a) near zero (b) very high (c) unchanged (d) negative
3. An LDR is unsuitable for high-speed counting because it is
(a) too small (b) slow to settle (c) too cheap (d) digital
4. Latching is needed because
(a) the buzzer is quiet (b) a fast event could be missed between loops (c) the LCD is slow (d) the laser flickers
5. An intruder passing with no alarm raised is a
(a) false positive (b) false negative (c) latch (d) systematic error
6. Real beam sensors resist being fooled by a torch because the beam is
(a) brighter (b) modulated (c) invisible (d) reflected

B. Fill in the blanks

- 1 A laser's narrow parallel beam is described as _____.
- 2 The light-sensitive track in an LDR is made of _____ sulphide.
- 3 The condition `&& !alarmLatched` stops one interruption being counted _____ times.
- 4 Ignoring a switch's contact chatter is called _____.

C. Answer in one or two lines

- 1 Write down your four measured LDR readings and justify the threshold you chose.
- 2 Explain why the buzzer is driven with `millis()` rather than `delay()`.
- 3 Describe how beam modulation would defeat the torch attack.
- 4 State your false positive and false negative counts and what they tell you.

UNIT 4

Intermediate Project: Ultrasonic Measurement and Calibration

Sessions 9, 10 and 11

This unit has no glamorous output. It produces a table, a graph and two correction numbers - and it is the most genuinely professional work in the book. Every instrument you will ever trust has been through this process.

By the end of this unit you will be able to:

- Explain how a piezoelectric transducer both emits and detects sound.
- Characterise a sensor against a reference standard across its range.
- Implement mean and median filters and show which is better, with data.
- Derive and apply a gain and offset correction.

SESSION 9

one class

Inside an Ultrasonic Transducer

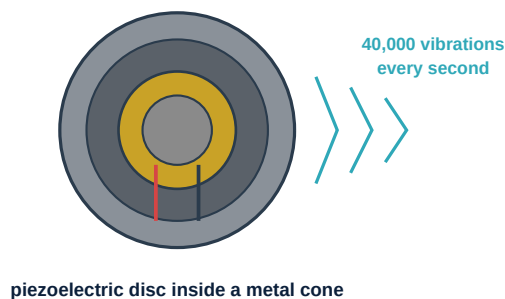
Today you will

- Learn how a piezoelectric crystal works in both directions.
- Understand beam angle, blind zone and maximum range.
- Mount the sensor for repeatable measurements.

Behind each metal grille on an HC-SR04 is a thin disc of piezoelectric ceramic bonded to a brass plate.

Piezoelectric means the material physically changes shape when a voltage is applied - and, remarkably, the reverse is also true: squeeze it and it generates a voltage.

A crystal that changes shape



Apply a voltage

the crystal flexes and pushes the air - it is a speaker

Sound hits it

the crystal is squeezed and makes a voltage - it is a microphone

40 kHz is chosen

high enough to be silent to us, low enough to travel usefully far

The two cylinders on an HC-SR04 are the same part used in opposite directions: one pushes air, one listens.

The beam spreads about 15 degrees, so a narrow pole may be missed while a wall behind it is seen instead.

Figure - Inside the HC-SR04: one crystal, two jobs

So the two cylinders are the same component used in opposite directions. Drive one at 40,000 cycles per second and it flexes 40,000 times a second, pushing the air - a speaker. Let the returning sound squeeze the other and it produces a tiny alternating voltage - a microphone. The module's own circuit amplifies that, decides when the echo arrived, and reports the answer as the width of a single pulse on the ECHO pin.

Behind the Scenes

40 kHz is not an arbitrary choice. It must be above roughly 20 kHz so humans cannot hear it. It must not be far above, because higher frequencies are absorbed by air much more strongly and the range would collapse. 40 kHz sits at a comfortable compromise - and it is also why dogs and some insects can hear your sensor perfectly well even though you cannot.

Property	Typical value	What it means for your measurements
Frequency	40 kHz	Silent to us; audible to some animals
Beam angle	about 15 degrees	A narrow pole may be missed and a wall seen behind it
Blind zone	below about 3 cm	The transducer is still ringing; readings are meaningless
Maximum range	about 4 metres	Beyond this the echo is too weak to detect
Settling time	about 60 ms	Measure faster and you may catch the previous echo

Watch Out

The blind zone catches everybody. Place an object 2 cm from the sensor and you will not get 2 cm - you will get nonsense, because the transmitting disc is still vibrating when the echo arrives. Your calibration must therefore start at 10 cm, and your final report must state the working range honestly rather than pretending the sensor works everywhere.

SESSION 10

one class

Characterising Your Own Sensor

Today you will

- Set up a repeatable measurement rig.
- Record readings against a tape across the full range.
- Tabulate error and identify its pattern.

This session produces data, not a demonstration. Fix the sensor at one end of a bench with tape marks at known distances. The target must be a flat, hard surface held square to the beam - a book or a clipboard is ideal, a person's hand is not, because it is neither flat nor repeatable.

Characterise.ino - raw, unfiltered, uncorrected

```
int trig = 9, echo = 10;

void setup() {
  pinMode(trig, OUTPUT);
  pinMode(echo, INPUT);
  Serial.begin(9600);
}

float readOnce() {
  digitalWrite(trig, LOW);
  delayMicroseconds(2);
  digitalWrite(trig, HIGH);
  delayMicroseconds(10);
  digitalWrite(trig, LOW);
  long t = pulseIn(echo, HIGH, 30000); // 30 ms timeout
  if (t == 0) { return -1; }           // nothing came back
  return t * 0.0343 / 2.0;             // cm, at about 20 degrees C
}

void loop() {
  float d = readOnce();
  Serial.println(d);
  delay(200);
}
```

Note 0.0343 rather than 0.034. The speed of sound in air is about 343 metres per second at 20 degrees Celsius - and it rises by roughly 0.6 metres per second for every degree warmer. On a 35 degree afternoon sound travels about 2.5 percent faster than your formula assumes, which at two metres is an error of five centimetres. Temperature is a genuine source of systematic error in every ultrasonic instrument.

Air temperature	Speed of sound	cm per microsecond	Error at 2 m if you assume 20 C
0 C	331 m/s	0.0331	reads about 3.6 percent short
20 C	343 m/s	0.0343	none - this is the assumption
30 C	349 m/s	0.0349	reads about 1.7 percent long
40 C	355 m/s	0.0355	reads about 3.5 percent long, roughly 7 cm

True distance (cm)	Reading 1	Reading 2	Reading 3	Mean	Error
10	_____	_____	_____	_____	_____
20	_____	_____	_____	_____	_____
40	_____	_____	_____	_____	_____
60	_____	_____	_____	_____	_____
80	_____	_____	_____	_____	_____
100	_____	_____	_____	_____	_____
150	_____	_____	_____	_____	_____
200	_____	_____	_____	_____	_____

Try This

Repeat the 60 cm measurement with the target square to the beam, and then tilted at about 30 degrees. A tilted surface reflects the sound away instead of back, and your reading will jump or vanish. Record what happens - it belongs in your limitations section, and it is exactly why a robot can drive into an angled wall it never saw.

SESSION 11

one class

Filtering, Correction and Error Analysis**Today you will**

- Plot your data and identify gain and offset error.
- Implement mean and median filters and compare them.
- Apply a correction and prove it worked.

Plot true distance on the horizontal axis and your reading on the vertical. A perfect sensor would give a straight line at 45 degrees passing through the origin. Yours will not - and how it deviates tells you which of two different faults you have.

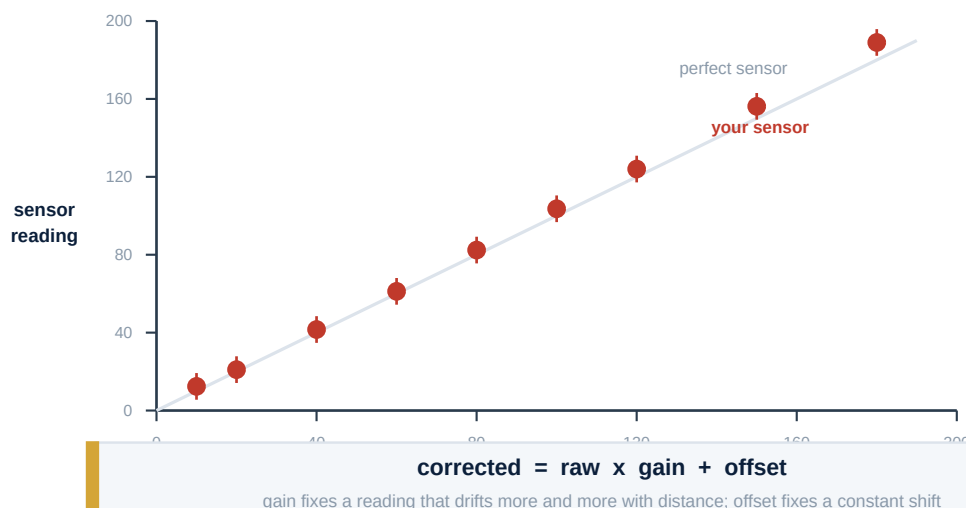
Characterising your sensor

Figure 4.1 - Plot it, and the error stops being a mystery

What the plot shows	Name of the fault	The correction
Line is parallel to perfect but shifted up or down	Offset error	Subtract a constant

What the plot shows	Name of the fault	The correction
Line starts correct but diverges more and more	Gain error	Multiply by a constant
Points scatter widely around the line	Random error	Filter - averaging or median
Line bends	Non-linearity	Beyond this course - state it as a limitation

Deriving the correction needs only two of your data points, taken far apart - say 20 cm and 150 cm. The **gain** is the ratio of the true span to the measured span. The **offset** is whatever remains after applying the gain.

Deriving gain and offset from two points

```
// Worked example. Suppose at a true 20 cm you read 21.0,
// and at a true 150 cm you read 156.0.
//
// gain  = (150 - 20) / (156.0 - 21.0) = 130 / 135.0 = 0.963
// offset = 20 - (21.0 * 0.963)         = 20 - 20.22  = -0.22

float gain  = 0.963;      // replace with YOUR numbers
float offset = -0.22;

float corrected(float raw) {
    return raw * gain + offset;
}
```

11.1 Two Filters, and Why the Median Usually Wins

An **averaging filter** adds several readings and divides. It is simple and it does reduce scatter. But it has a weakness: a single wild reading - an echo bounced off a chair leg - drags the average with it.

A **median filter** sorts the readings and takes the middle one. A wild value sorts to the end and is simply ignored. For sensors whose errors are occasional and large - which describes ultrasonic sensors exactly - the median is almost always the better choice.

A five-sample median filter

```
float readMedian() {
    float v[5];
    for (int i = 0; i < 5; i++) {
        v[i] = readOnce();
        delay(60);                // respect the settling time
    }

    for (int i = 0; i < 4; i++) {    // a simple sort
        for (int j = i + 1; j < 5; j++) {
            if (v[j] < v[i]) {
                float temp = v[i];
                v[i] = v[j];
                v[j] = temp;
            }
        }
    }

    return corrected(v[2]);        // the middle value, then corrected
}
```

At a true 60 cm	Raw	Mean of 5	Median of 5
Highest reading seen	_____	_____	_____
Lowest reading seen	_____	_____	_____

At a true 60 cm	Raw	Mean of 5	Median of 5
Spread, highest minus lowest	_____	_____	_____
Which is tightest?			

Challenge

Deliberately introduce a wild reading during a run - wave a book across the beam once. Record what the mean does and what the median does. That single experiment is the most convincing paragraph you can put in your report.

New word	What it means
Piezoelectric	Changing shape under voltage, and producing voltage when squeezed.
Beam angle	The cone within which the sensor can see.
Blind zone	The close range where the sensor cannot measure at all.
Gain error	Error that grows in proportion to the reading.
Offset error	A constant shift in every reading.
Median filter	Taking the middle of several sorted readings.

CHECK WHAT YOU LEARNT

Check Yourself - Unit 4**A. Choose the correct answer**

1. A piezoelectric disc can act as
(a) only a speaker (b) only a microphone (c) both, in opposite directions (d) neither
2. 40 kHz is chosen because it is
(a) the loudest (b) just above human hearing (c) the cheapest (d) visible
3. An object 2 cm from the sensor gives nonsense because of the
(a) beam angle (b) blind zone (c) gain error (d) timeout
4. A reading that drifts further out as distance grows indicates
(a) offset error (b) gain error (c) random error (d) no error
5. A single wild reading affects
(a) the median more than the mean (b) the mean more than the median (c) both equally (d) neither
6. Sound travels faster when the air is
(a) cooler (b) warmer (c) drier (d) still

B. Fill in the blanks

- 1 The beam of an HC-SR04 spreads about _____ degrees.
- 2 Readings below about _____ cm fall inside the blind zone.
- 3 A constant shift in every reading is called _____ error.
- 4 The median filter works by _____ the readings and taking the middle one.

C. Answer in one or two lines

- 1 Write down your gain and offset, and the two data points you derived them from.
- 2 Explain why a tilted target gives a poor or missing reading.
- 3 State the working range of your sensor and justify it from your data.
- 4 Explain, with your own numbers, why you chose a median or a mean filter.

UNIT 5

Capstone Project: Maze Solver Robot

Sessions 12, 13, 14 and 15

The capstone brings everything together: three sensors, a driver stage, a state machine, and memory. The robot must reach the exit of a maze it has never seen, and then do it again faster using what it learnt on the first run.

By the end of this unit you will be able to:

- Implement and document a navigation algorithm end to end.
- Design a state machine and explain why it beats a chain of if statements.
- Store a route in memory and simplify it.
- Defend design trade-offs with test data in a technical viva.

SESSION 12

one class

Specification, Algorithms and the Left-Hand Rule

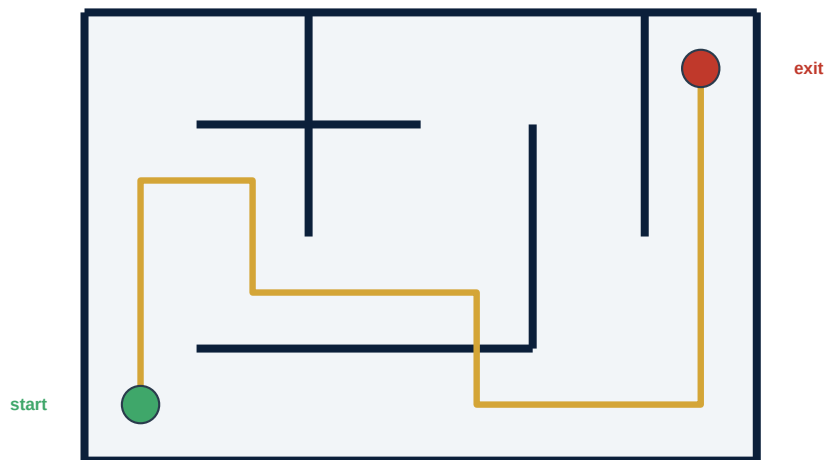
Today you will

- Write the requirement specification for the capstone.
- Understand the left-hand rule and its limits.
- Allocate team roles and plan the build.

Solving a maze sounds like it needs intelligence. It does not. A rule discovered centuries before computers existed will do it: keep one hand on the wall and walk. You will trace every wall connected to the one you started on, and if the exit is on that same connected wall, you will certainly reach it.

The left-hand rule

Keep your left hand on the wall and walk. You will always reach the exit.



It is not the shortest route - but it never fails, and it needs no map at all.

Figure 5.1 - A rule so simple a robot with three sensors can follow it

Behind the Scenes

The left-hand rule has an honest limitation, and stating it is part of good engineering. If the maze contains a loop that is not connected to the outer wall, and the exit sits inside that loop, a wall-follower will circle forever. Mazes with this property are called **non-simply-connected**. Competition mazes are usually designed to be solvable by wall-following - but you should say so in your report rather than claim your robot solves any maze.

Priority	If this is possible, do it	Why this order
1	Turn left	The rule: always prefer the left-hand wall
2	Go straight	Only when left is blocked
3	Turn right	Only when both left and straight are blocked
4	Turn around	A dead end - the last resort

Write your specification before Session 13. Four testable requirements is enough, and here is the shape they should take.

#	Requirement	How it will be tested
R1	The robot shall reach the exit of a 2 x 2 m maze without human intervention.	Three consecutive runs, no rescues
R2	The robot shall not touch a maze wall during a run.	Observation by a second group
R3	The second run shall be faster than the first.	Stopwatch, both runs recorded
R4	A run shall complete within _____ seconds.	Stopwatch

Role	Responsible for	Name
Systems lead	The specification, the test plan and the final report	_____
Mechanical	Chassis, sensor mounting height and spacing, wiring discipline	_____
Software	State machine, path storage, tuning	_____
Test engineer	Running trials, recording every result including failures	_____

SESSION 13

one class

Subsystem Build and Independent Testing**Today you will**

- Build the drive base and mount three IR sensors.
- Test each subsystem alone before integrating.
- Verify the direction table for both motors.

The single most valuable habit in this session is **independent testing**. Build the drive base and prove it drives. Then mount the sensors and prove they read correctly, with the motors switched off. Only then combine them. Integrate two untested subsystems and any fault has four possible homes instead of one.

How an IR sensor tells black from white

It shines invisible infrared light down and watches how much bounces back

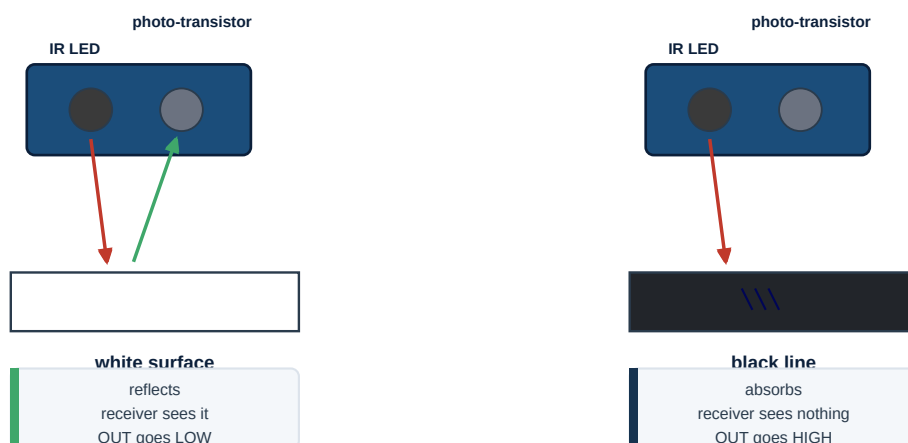


Figure 5.1 - Two tiny components, one very useful trick

Each IR module carries an infrared LED and a phototransistor. White scatters the light back and the output goes LOW; black absorbs it and the output goes HIGH. Because the sensor supplies its own illumination, it works identically in a dark room and a bright one - though direct sunlight can still saturate it.

13.1 The Power Stage

An Arduino output pin is a signalling device, not a power supply. It can deliver about 20 milliamps safely; a BO motor starting under load can demand several hundred. Connect one directly and the pin is destroyed permanently.

The L298N contains two **H-bridges**, each a set of four electronic switches arranged around a motor in the shape of the letter H. Close one diagonal pair and current flows one way through the motor; close the other pair and it flows the other way. The Arduino only ever sends tiny logic signals telling those switches what to do.

The L298N: two switches decide which way a motor turns

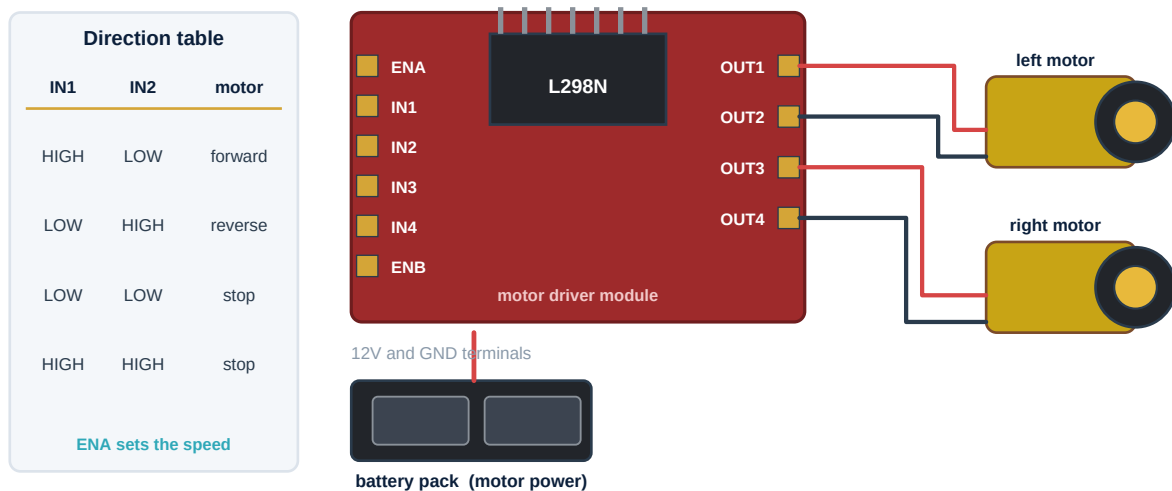


Figure 5.1 - IN pins choose the direction, EN pins choose the speed

Look at your BO motor too. The silver cylinder at the back is the motor itself, spinning several thousand times a minute with almost no strength. The plastic housing in front is a **gearbox** that trades that speed for torque - gear down by fifty and you turn fifty times slower with roughly fifty times the turning force. That is why the wheel runs at a usable 100 rpm and can actually push the chassis, battery and boards along the floor.

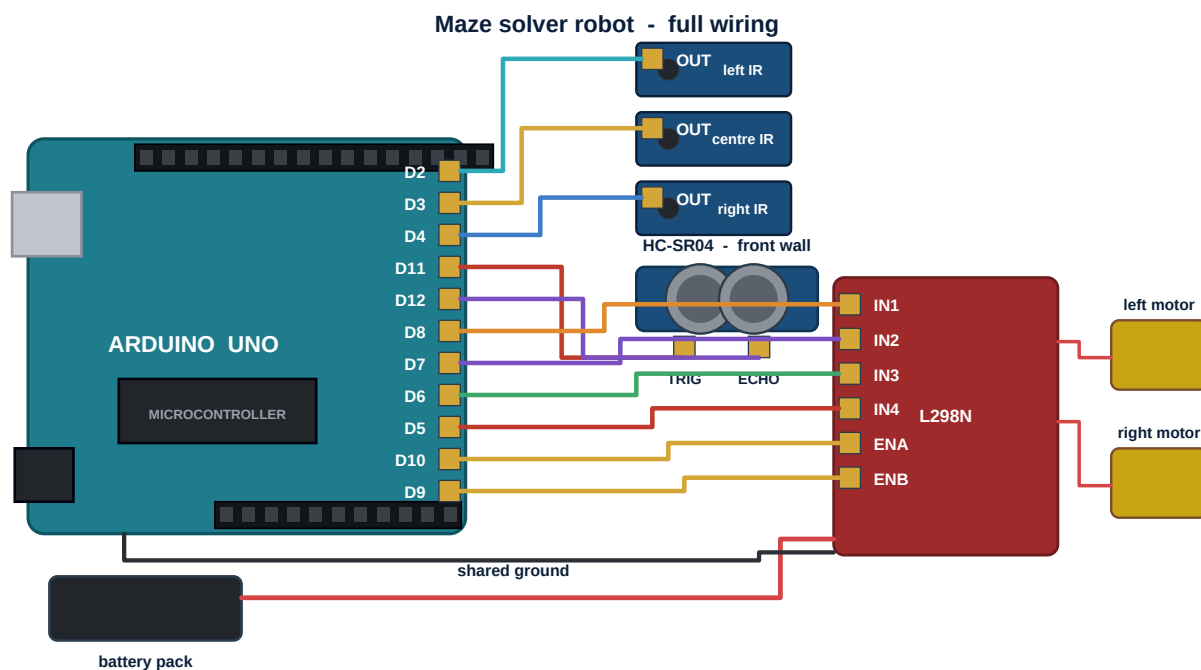


Figure 5.2 - Three line sensors, one wall sensor, two motors

Connection	Arduino pin	Note
Left IR OUT	D2	Mounted about 3 cm left of centre
Centre IR OUT	D3	Directly on the line
Right IR OUT	D4	About 3 cm right of centre
HC-SR04 TRIG / ECHO	D11 / D12	Front wall detection
L298N IN1 / IN2	D8 / D7	Left motor direction
L298N IN3 / IN4	D6 / D5	Right motor direction

Connection	Arduino pin	Note
L298N ENA / ENB	D10 / D9	PWM speed
L298N GND	Arduino GND	Shared ground - mandatory
L298N 12V / GND	Battery + / -	Motor supply, never from the Arduino

Watch Out

Remove the ENA and ENB jumper caps before wiring them, or the motors will run at full speed and ignore every speed value you write. And check the sensor mounting height: 5 to 10 mm above the floor. Too high and the reflection is too weak; too low and the sensor sees only its own glow.

SESSION 14

one class

State Machines and Path Memory**Today you will**

- Design the robot's behaviour as a state machine.
- Store the route taken as a sequence of turns.
- Run the maze for the first time.

A long chain of if statements becomes unmanageable once a machine has to remember what it was doing. The professional structure is a **state machine**: the robot is always in exactly one named state, each state has clear behaviour, and specific events move it to another state.

State	What the robot does	What moves it on
FOLLOWING	Steers to keep the centre sensor on the line	A junction is detected
AT_JUNCTION	Stops, reads all three sensors and the front wall	A decision is made
TURNING	Executes the chosen turn for a tuned time	The turn completes
DEAD_END	Turns around 180 degrees	The turn completes
FINISHED	Stops and reports the stored path	Nothing - the run is over

The memory is simpler than students expect. Every turn is recorded as a single character: L for left, R for right, S for straight, B for back. A route becomes a short string such as LBLLRS. Then comes the elegant part - that string can be **simplified**. Any sequence containing a B represents a dead end that was entered and left, and can be replaced by the single turn that would have avoided it.

What was walked	Becomes	In plain English
LBR	B	Went left, dead end, came back right - it was a dead end straight ahead
LBS	R	Left, dead end, straight on - should have turned right
RBL	B	Right, dead end, left - dead end ahead
SBS	B	Straight, dead end, straight - turn around
LBL	S	Left, dead end, left - should have gone straight

MazeSolver.ino - the state machine and the path memory

```
// ----- states -----
#define FOLLOWING 0
#define AT_JUNCTION 1
#define TURNING 2
#define FINISHED 3
```

```

int state = FOLLOWING;

char path[40];           // the route, one character per turn
int  pathLength = 0;     // watch your SRAM - 40 is plenty

int leftIR = 2, centreIR = 3, rightIR = 4;
int in1 = 8, in2 = 7, in3 = 6, in4 = 5, ena = 10, enb = 9;
int speed = 120;

void recordTurn(char c) {
    if (pathLength < 39) {
        path[pathLength] = c;
        pathLength = pathLength + 1;
    }
}

void simplify() {        // collapse a dead end into one turn
    if (pathLength < 3) { return; }
    if (path[pathLength - 2] != 'B') { return; }

    char a = path[pathLength - 3];
    char c = path[pathLength - 1];
    char result = 'S';

    if (a == 'L' && c == 'R') { result = 'B'; }
    if (a == 'L' && c == 'S') { result = 'R'; }
    if (a == 'R' && c == 'L') { result = 'B'; }
    if (a == 'S' && c == 'S') { result = 'B'; }
    if (a == 'L' && c == 'L') { result = 'S'; }

    pathLength = pathLength - 3;
    recordTurn(result);
}

void loop() {
    int L = digitalRead(leftIR);
    int C = digitalRead(centreIR);
    int R = digitalRead(rightIR);

    switch (state) {

        case FOLLOWING:
            followLine(L, C, R);
            if (L == HIGH || R == HIGH) { state = AT_JUNCTION; }
            break;

        case AT_JUNCTION:
            stopMotors();
            delay(120);           // let the chassis settle
            if (L == HIGH)        { recordTurn('L'); turnLeft(); }
            else if (C == HIGH)   { recordTurn('S'); goStraight(); }
            else if (R == HIGH)   { recordTurn('R'); turnRight(); }
            else                  { recordTurn('B'); turnAround(); }
            simplify();
            state = TURNING;
            break;

        case TURNING:
            if (C == HIGH) { state = FOLLOWING; } // back on the line
            break;

        case FINISHED:
            stopMotors();
            break;
    }
}

```

```
}
}
```

Behind the Scenes

Notice that `simplify()` runs after every turn, not once at the end. That keeps the stored path short as you go, which matters when you only have two kilobytes of SRAM. It is a small design decision with a real engineering reason behind it - exactly the kind of thing a viva examiner will ask you to justify.

SESSION 15

one class

Optimisation, Final Runs and Technical Viva

Today you will

- Use the stored path to run the maze a second time, faster.
- Complete the acceptance tests and record the data.
- Defend your design in a technical viva.

Once the exit is reached, the simplified path holds the direct route with every dead end removed. On the second run the robot no longer needs to think at each junction - it simply reads the next character and executes it. That is why the second run is faster, and it is the requirement R3 you wrote in Session 12.

The second run: no decisions, just recall

```
int step = 0;

void fastRun() {
  if (atJunction()) {
    char next = path[step];
    step = step + 1;

    if (next == 'L') { turnLeft(); }
    else if (next == 'R') { turnRight(); }
    else if (next == 'S') { goStraight(); }

    if (step >= pathLength) { state = FINISHED; }
  } else {
    followLine(digitalRead(leftIR), digitalRead(centreIR), digitalRead(rightIR));
  }
}
```

What you see	Most likely cause	What to do
Nothing happens	Missing Arduino-to-L298N ground	Add it. The commonest fault.
Speed value ignored	ENA and ENB caps still fitted	Remove both
Turns overshoot or undershoot	Turn duration not tuned for this floor	Adjust in 50 ms steps, retest three times each
Junction detected twice	Sensor still over the marking after the turn	Add a short delay after a turn completes
Path grows and never simplifies	The B character is not being recorded	Print the path after every turn and watch it
Robot drifts on straights	Motors mismatched	Trim one motor's PWM by 5 to 10

Acceptance test	Result	Tick
R1 Reaches the exit unaided, three consecutive runs	_____	■
R2 No wall contact during a run	_____	■

Acceptance test	Result	Tick
R3 Second run measurably faster than the first	_____	■
R4 Run completes within the stated time	_____	■
Stored path simplifies correctly, printed and checked	_____	■

15.1 The Technical Viva

You will be asked to defend your design, not merely demonstrate it. Prepare honest answers to these, with data where data exists.

- 1 Why did you choose the left-hand rule, and what kind of maze would defeat it?
- 2 Show the state machine and explain why you did not use a chain of if statements.
- 3 Explain one path simplification rule using a route your robot actually walked.
- 4 What is your robot's slowest and fastest recorded run, and what accounts for the difference?
- 5 Which subsystem failed most often during testing, and how did you diagnose it?
- 6 If you had one more session, what would you change, and what evidence supports that choice?

Challenge

Add the HC-SR04 as a genuine safety layer. Before executing any forward movement, check the distance ahead and refuse to move if a wall is closer than 8 cm. Log every refusal to the serial line. Real autonomous systems all have a layer like this, and it is called a safety interlock.

New word	What it means
State machine	A design where the system is always in one named state.
Left-hand rule	Always prefer the leftmost available turn.
Path simplification	Collapsing a dead-end detour into the single turn that avoids it.
Safety interlock	A layer that refuses an unsafe action regardless of the main logic.
Independent testing	Proving each subsystem alone before integrating.

CHECK WHAT YOU LEARNT

Check Yourself - Unit 5**A. Choose the correct answer**

- The left-hand rule fails on a maze that is
(a) large (b) non-simply-connected (c) dark (d) square
- In a state machine the system is always in
(a) several states (b) exactly one state (c) no state (d) a loop
- The path LBR simplifies to
(a) L (b) B (c) R (d) S
- simplify() is called after every turn in order to
(a) speed up the motors (b) keep the stored path short (c) improve the sensors (d) reduce current
- The second run is faster because the robot
(a) drives faster (b) no longer decides at junctions (c) has fewer sensors (d) uses less power
- A layer that refuses an unsafe action regardless of the main logic is a
(a) state machine (b) safety interlock (c) median filter (d) bootloader

B. Fill in the blanks

- Each turn is stored as a single _____: L, R, S or B.
- The four states in this design are FOLLOWING, AT_JUNCTION, _____ and FINISHED.
- Proving each subsystem alone before combining them is called _____ testing.
- IR sensors should be mounted _____ to _____ mm above the floor.

C. Answer in one or two lines

- Explain the left-hand rule and state honestly when it fails.
- Write out a route your robot walked and show one simplification step.
- Explain why a state machine is better than a chain of if statements here.
- Give your first-run and second-run times, and account for the difference.

Extra Challenges

For groups whose builds are working early. All use parts you already have.

#	Challenge	Uses
1	Modulate the laser and accept only light pulsing at that rate	laser + LDR + timing
2	Timestamp every intrusion and show the last three on the LCD	millis() + array
3	An armed and disarmed mode, toggled by the push switch	state machine
4	Log 50 readings at each of five distances and report the standard deviation	HC-SR04 + maths
5	Compensate for air temperature using the DHT11 formula	speed of sound
6	Plot live distance on the Serial Plotter, raw against filtered	Serial Plotter
7	A servo-mounted sensor that sweeps and reports the clearest direction	HC-SR04
8	Print the simplified path to the LCD at the end of a run	LCD + array
9	A safety interlock that stops the robot before any wall contact	HC-SR04
10	Compare left-hand and right-hand rules on the same maze and report which won	test data

MIXED REVISION

Twenty Questions to Test Yourself

- 1 Write a testable requirement for an alarm's response time.
- 2 Distinguish accuracy from precision with an example of each.
- 3 Which kind of error can be corrected in software, and which must be filtered?
- 4 How would you estimate the uncertainty of a measurement from data alone?
- 5 Which chip on the Arduino Uno is the actual computer, and what clocks it?
- 6 How much flash and how much SRAM does it have, and what lives in each?
- 7 Describe the four stages between pressing Upload and your program running.
- 8 How many distinct values can a 10-bit ADC report, and what is one step worth?
- 9 Explain successive approximation in one sentence.
- 10 Why is `delay()` unacceptable in a security system?
- 11 How does a laser diode differ from an LED in the light it produces?
- 12 Explain photoconductivity in an LDR.
- 13 Why does latching matter for a fast beam-break event?
- 14 How does a piezoelectric disc work as both speaker and microphone?
- 15 State the beam angle, blind zone and maximum range of an HC-SR04.
- 16 How is a gain correction derived from two data points?
- 17 Why does a median filter beat an averaging filter for ultrasonic data?
- 18 State the left-hand rule and one maze that defeats it.
- 19 Explain the simplification LBS becomes R.
- 20 Name the design decision in your capstone you are least confident about, and why.

Wiring Reference - All Three Projects

Keep this page open while you build. Every pin used this year, in one place.

Project 1 - Laser Security System

Arduino pin	Connects to
A0	LDR module AO
D8	Buzzer +
D7	Push switch, with 10K to GND
D12 / D11	LCD RS / E
D5, D4, D3, D2	LCD D4, D5, D6, D7
-	Laser + to 5V through 220 ohm, - to GND
-	LCD V0 to potentiometer wiper; VSS, RW, K to GND; VDD to 5V

Project 2 - Ultrasonic Measurement and Calibration

Arduino pin	Connects to
D9	HC-SR04 TRIG
D10	HC-SR04 ECHO
D12 / D11	LCD RS / E
D5, D4, D3, D2	LCD D4, D5, D6, D7
5V and GND	HC-SR04 VCC and GND
-	LCD V0 to potentiometer wiper

Project 3 - Maze Solver Robot

Arduino pin	Connects to
D2 / D3 / D4	Left / centre / right IR sensor OUT
D11 / D12	HC-SR04 TRIG / ECHO
D8, D7	L298N IN1, IN2 - left motor direction
D6, D5	L298N IN3, IN4 - right motor direction
D10, D9	L298N ENA, ENB - speed
GND	L298N GND - the shared ground, never omit
-	L298N 12V and GND to the battery pack
-	L298N OUT1-4 to the two motors

Words to Know

Word	Meaning
Accuracy	How close a reading is to the true value.
ADC	The circuit that converts a voltage into a number.
ATmega328P	The microcontroller that is the real computer on the board.
Beam angle	The cone within which an ultrasonic sensor can see.
Blind zone	The close range where a sensor cannot measure at all.
Bootloader	A resident program that writes your code into flash.
Collimated	Formed into a narrow parallel beam.
Debounce	Ignoring the brief contact chatter of a switch.
False positive	An alarm with nothing to alarm about.
False negative	No alarm when there really was something.
Gain error	Error that grows in proportion to the reading.
H-bridge	Four switches that let a circuit reverse a motor.
Independent testing	Proving each subsystem alone before integrating.
Latching	Staying on once triggered, until deliberately cleared.
Left-hand rule	Always prefer the leftmost available turn.
Median filter	Taking the middle of several sorted readings.
Modulation	Pulsing a signal at a known rate so it cannot be imitated.
Non-blocking	Code that waits without halting everything else.
Offset error	A constant shift present in every reading.
Photoconductivity	Light freeing electrons and lowering resistance.
Piezoelectric	Changing shape under voltage, and producing voltage when squeezed.
Precision	How closely repeated readings agree with each other.
Random error	Unpredictable scatter, reducible by averaging.
Requirement specification	A written, testable statement of what a system must do.
Resolution	The smallest change a measuring system can detect.
Safety interlock	A layer that refuses an unsafe action regardless of the main logic.
Shared ground	The wire letting two supplies agree on what zero volts means.
State machine	A design where the system is always in exactly one named state.
Successive approximation	The binary-search method an ADC uses.
Systematic error	A consistent offset or drift that can be corrected.
Uncertainty	The range within which the true value very likely lies.

Answer Key

Section A only. Sections B and C are for you and your trainer to discuss - there is often more than one defensible answer.

Check Yourself - Unit 1

Q	Answer	The correct option
1	(b)	testable
2	(b)	systematic error
3	(b)	precision
4	(b)	averaging several readings
5	(b)	40 and 44
6	(b)	13.6 mA

Check Yourself - Unit 2

Q	Answer	The correct option
1	(b)	the ATmega328P
2	(b)	SRAM
3	(c)	1024 values
4	(b)	4.9 millivolts
5	(b)	successive approximation
6	(c)	unsigned long

Check Yourself - Unit 3

Q	Answer	The correct option
1	(b)	coherent and collimated
2	(b)	very high
3	(b)	slow to settle
4	(b)	a fast event could be missed between loops
5	(b)	false negative
6	(b)	modulated

Check Yourself - Unit 4

Q	Answer	The correct option
1	(c)	both, in opposite directions
2	(b)	just above human hearing
3	(b)	blind zone
4	(b)	gain error
5	(b)	the mean more than the median
6	(b)	warmer

Check Yourself - Unit 5

Q	Answer	The correct option
1	(b)	non-simply-connected
2	(b)	exactly one state
3	(b)	B
4	(b)	keep the stored path short
5	(b)	no longer decides at junctions
6	(b)	safety interlock

My Engineering Log

Fill one row after every class, including the sessions where nothing worked. A log of failures and their diagnoses is assessed alongside your hardware.

Session	What I built or measured	What failed	Diagnosis and fix
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			

A last word

Look back at what changed this year. You stopped asking whether a machine works and started asking how well, within what range, and with what uncertainty. You wrote requirements before you wired anything, and you recorded failures instead of hiding them. Those habits, far more than any single project, are what engineering actually is.